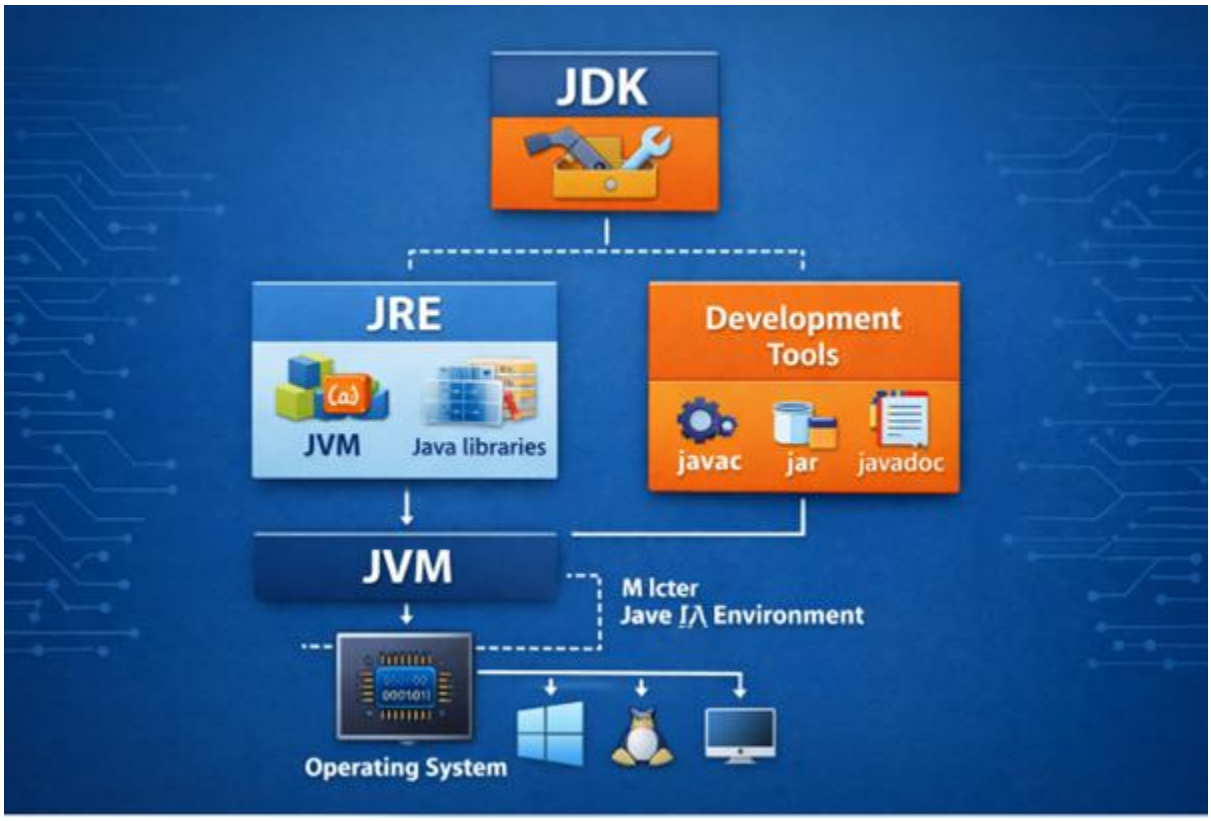
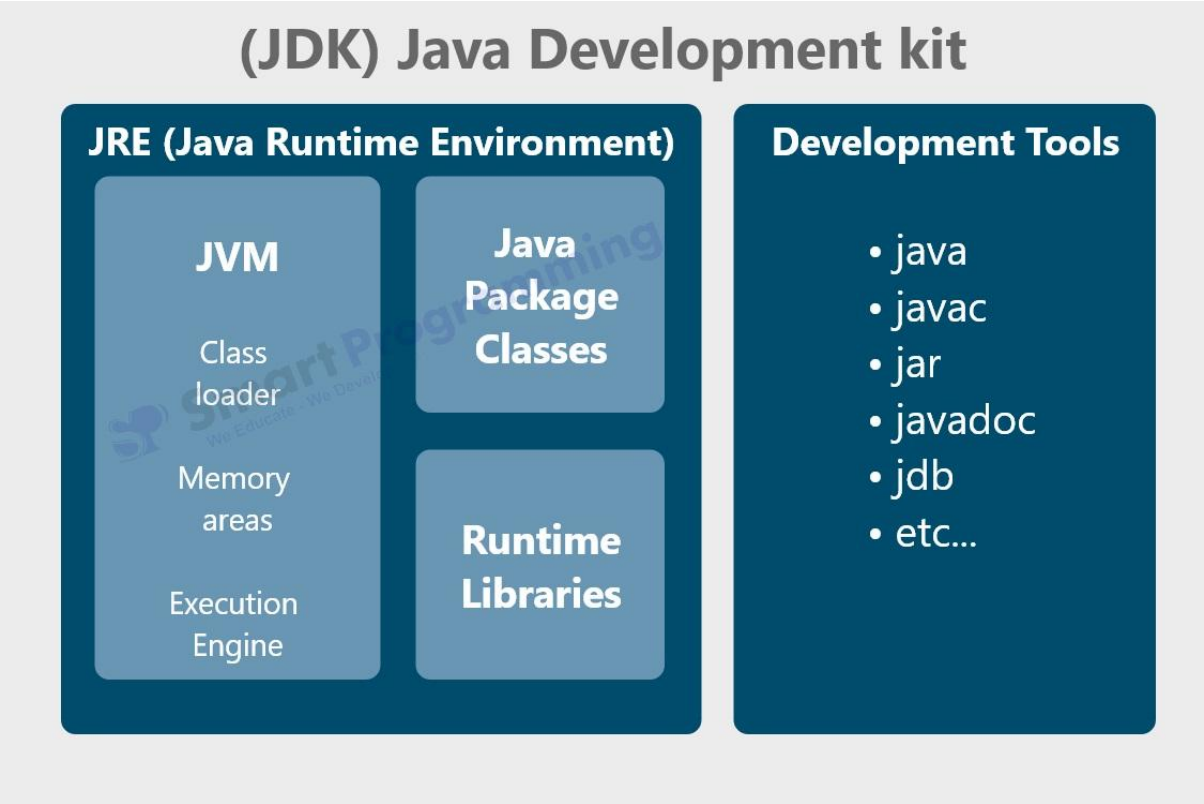
JDK , JVM , JRE Architecture



Java Architecture Overview



Relationship:

```
JDK
└── JRE
    └── JVM
```

Or

JDK = JRE + Development Tools
JRE = JVM + Libraries

1 JVM (Java Virtual Machine)

The Java Virtual Machine (JVM) is the core component of Java technology.

It is a virtual computer that runs Java programs.

Instead of converting Java programs directly into machine code, Java converts them into bytecode, which is executed by the JVM.

Because every operating system has its own JVM implementation, the same Java program can run on Windows, Linux, and macOS.

Responsibilities of JVM

The JVM performs many important tasks:

- Loads Java class files into memory
- Verifies bytecode for security and correctness
- Converts bytecode into machine code
- Executes Java programs
- Manages memory allocation
- Performs garbage collection

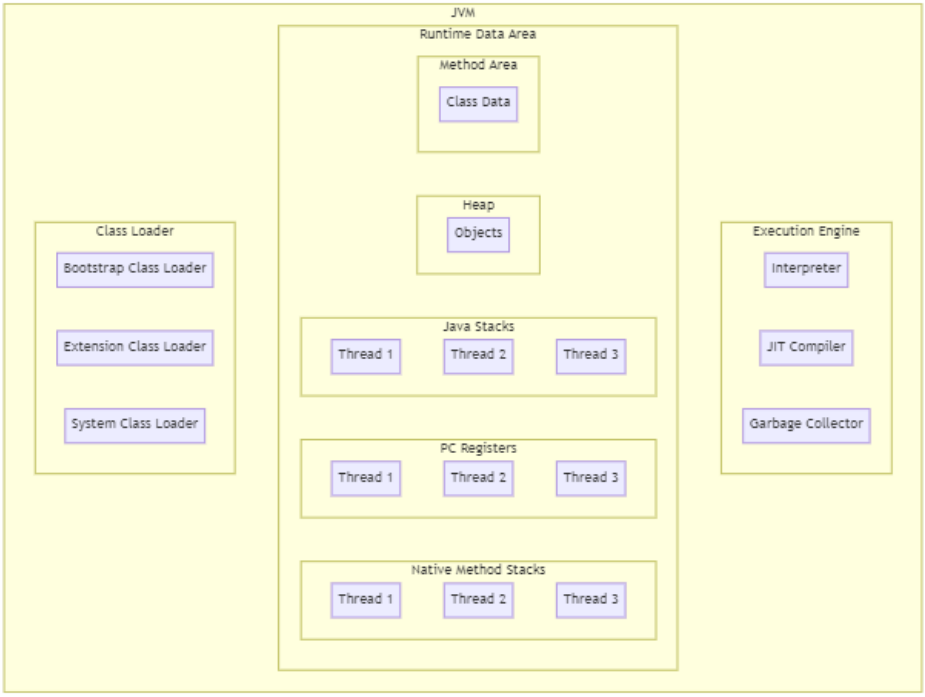
Java Program Execution Process

Java Source Code (.java)
↓
Compiler
(javac)
↓
Bytecode (.class)
↓
JVM
↓
Machine Code
↓
Operating System

Steps explained:

1. Developer writes Java code in .java file
2. The **javac compiler** converts it to **bytecode (.class)**
3. JVM loads the bytecode
4. JVM converts bytecode into machine code
5. Program executes on the operating system

JVM Internal Architecture :



1. Class Loader Subsystem

The Class Loader loads .class files into memory.

It performs three main steps:

Loading

The class loader loads Java class files into JVM memory.

Example:

Test.class

Linking

Linking includes:

1️⃣ **Verification** – checks if bytecode is valid
2️⃣ **Preparation** – allocates memory for static variables
3️⃣ **Resolution** – converts symbolic references to direct references

Initialization

Static variables and static blocks are initialized.

2. Runtime Data Areas

These areas manage memory during program execution.

Heap

Heap memory stores:

- Objects
- Instance variables

Example:

```
Student s = new Student();
```

The object Student is stored in heap memory.

Heap is shared among all threads.

Stack

Stack memory stores:

- Method calls
- Local variables

Example:

```
int a = 10;
```

Stored inside the stack frame.

Each thread has its own stack.

Method Area

Stores:

- Class structure
- Static variables
- Method code
- Constant pool

PC Register

The Program Counter Register stores the address of the current instruction being executed.

Each thread has its own PC register.

Native Method Stack

Used for executing native methods written in languages like C or C++.

Example:

JNI (Java Native Interface)

3️⃣ Execution Engine

The execution engine runs the bytecode.

It consists of:

Interpreter

The interpreter reads bytecode line by line and executes it.

Advantage:

• Simple execution

Disadvantage:

• Slower performance

JIT Compiler (Just-In-Time Compiler)

To improve performance, JVM uses JIT Compiler.

Instead of interpreting repeatedly, JIT converts frequently executed bytecode into native machine code.

Benefits:

• Faster execution
• Optimized performance

4️⃣ Garbage Collector (GC)

Java automatically manages memory using Garbage Collection.

When an object is no longer referenced, the garbage collector removes it from memory.

Example:

```
Student s = new Student();
s = null;
```

Now the object becomes eligible for garbage collection.

Advantages:

• Prevents memory leaks
• Automatic memory management
• Improves application stability

2️⃣ JRE (Java Runtime Environment)

The Java Runtime Environment (JRE) provides everything needed to run Java programs.

If a user only wants to run Java applications, they only need JRE.

Components of JRE

```
JRE
 ├── JVM
 ├── Java Class Libraries
 └── Supporting Files
```

Java Class Libraries

These are prewritten classes that developers use.

Common libraries include:

java.lang
java.util
java.io
java.net
java.sql

Example:

```
System.out.println("Hello Java");
```

System and PrintStream classes come from java.lang library.

Role of JRE

JRE ensures that Java applications run **consistently across different systems**.

This is what makes Java **platform independent**.

3️⃣ JDK (Java Development Kit)

The **JDK is the complete toolkit used by developers to build Java applications.**

It includes:

- JRE
- Development tools
- Compiler
- Debugging utilities

JDK Structure

```
JDK
 ├── JRE
 │    ├── JVM
 │    └── Libraries
 │
 └── Development Tools
```

Important JDK Tools

javac

Java compiler that converts .java files into .class.

Example:

```
javac Hello.java
```

---

java

Used to run the compiled program.

```
java Hello
```

---

jar

Packages multiple files into a **Java Archive file**.

Example:

jar cvf app.jar Test.class

---

javadoc

Generates documentation from Java source code.

---

jdb

Java debugger used to detect and fix errors.

Complete Java Development Flow

Write Code
↓
Save as .java file
↓
Compile using javac
↓
Generate .class bytecode
↓
JVM loads bytecode
↓
Execution Engine runs code
↓
Program runs on OS

Difference Between JDK, JRE, and JVM

Component Description

Aspect	JDK	JRE	JVM
Purpose	Used to develop Java applications	Used to run Java applications	Executes Java bytecode
Platform Dependency	Platform-dependent (OS specific)	Platform-dependent (OS specific)	JVM is OS-specific, but bytecode is platform-independent
Includes	JRE + Development tools (javac, debugger, etc.)	JVM + Libraries (e.g., rt.jar)	ClassLoader, JIT Compiler, Garbage Collector
Use Case	Writing and compiling Java code	Running a Java application on a system	Convert bytecode into native machine code

Simple Way to Remember

JDK → Used to DEVELOP Java programs
JRE → Used to RUN Java programs
JVM → Used to EXECUTE Java bytecode

Or

JDK = JRE + Tools
JRE = JVM + Libraries

Key Insight

Java programs do not run directly on the operating system.

Instead:

Java Code → Bytecode → JVM → Machine Code

This architecture is what makes Java:

- Platform Independent
- Secure
- Portable